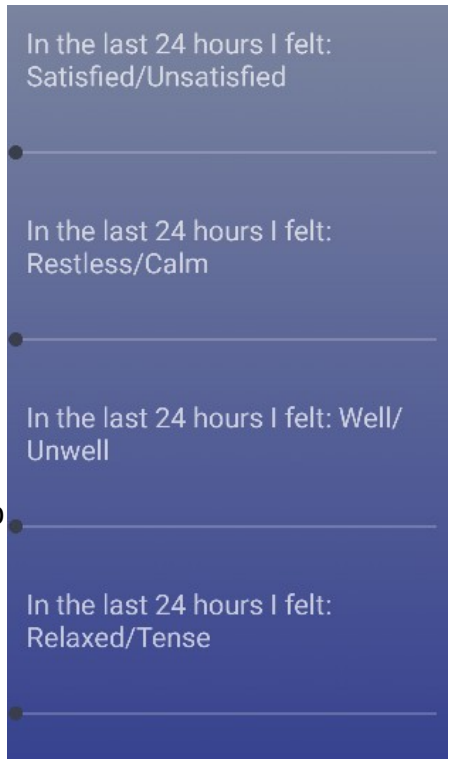
Overview of our App (4 Pages)

Our app, called „Larry‘s Liftin‘ Lab“ is a well-being and fitness-focused research app that allows users to monitor their own affective well-being and take steps to change it for the better. Its second functionality is that it is highly customizable and allows researchers who study affective well-being to use the app‘s data to gain insight into how different measures impact user‘s well-being. This is in accordance to the must-haves, should-haves and most could-haves that were provided to our group by the SE Lab team.

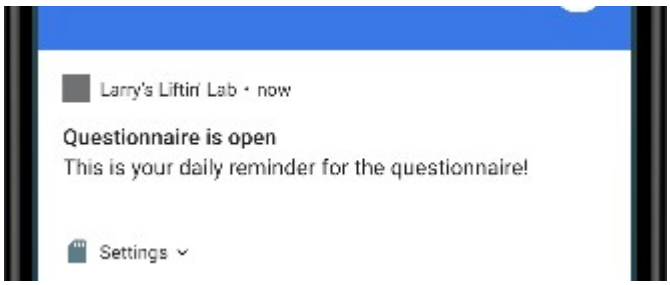
The feature of the app that is expected to generate the largest amount and the most valuable user data is the Questionnaire.



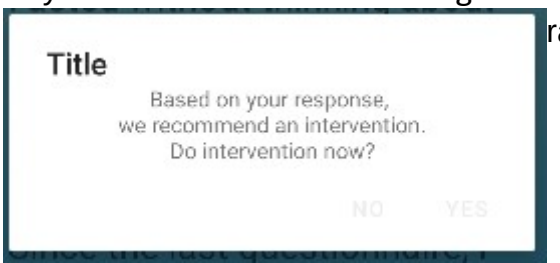
The Questionnaire allows the user to answer a few questions about their mood, their current situation and their thoughts on both. The results are saved in a local file and can later be viewed through the „Data“ fragment in the HomeActivity.

Some questions need to be answered through a slider (e.g. the questions displayed in the picture on the right), some are chocies, and some others depend on the answers to other questions. The Questionnaire also provides an EditText where the user can note down any additional comments on the Questionnaire or the time period since the last one. These comments could also be factored into the evaluation by the researchers, as they are saved alongside the rest of the questionnaire.

Whenever there is a questionnaire available, the user will receive a notification to their phone reminding them of this fact. This notification is clickable and will take the user directly to the relevant questionnaire. This questionnaire behaves exactly as if it was opened from the app, with the correct timeslot, and will open the “Digit Span Task” and an intervention if relevant (see below).



Based on an algorithm which was sadly not provided by the researchers, but which could be exchanged at any time by modifying a single method (which is the aptly named method from the QuestionnaireActivity `shouldDoInterventionAlgorithm`), the user may receive a suggestion to do a random intervention:



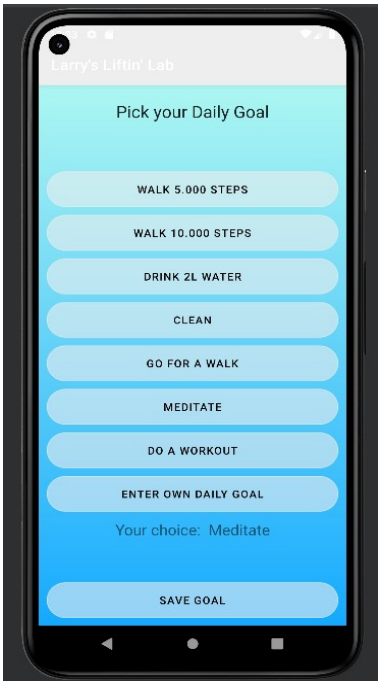
This allows researchers to monitor if doing an intervention improves successive well-being scores and thus if interventions are a good way to improve the well-being of people participating. The intervention is chosen at random currently, but this algorithm is easily interchangeable as well.

The Questionnaire is also sometimes (Concretely: Whenever it is done in the noon-timeslot) followed up by the ”Digit Span Task” (short: DST). The DST works as follows: On pressing start, a randomized sequence of numbers is shown one at a time. Then, the user must input the numbers again in the correct order. If the user is successful, the length is increased by one and the task repeats. As soon as the user fails, the length of the sequence is recorded and saved locally and the task ends. The next time the user starts the digit span task, this is used as the initial span length, which ensures that the user is always appropriately challenged in the DST.

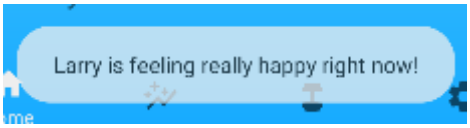


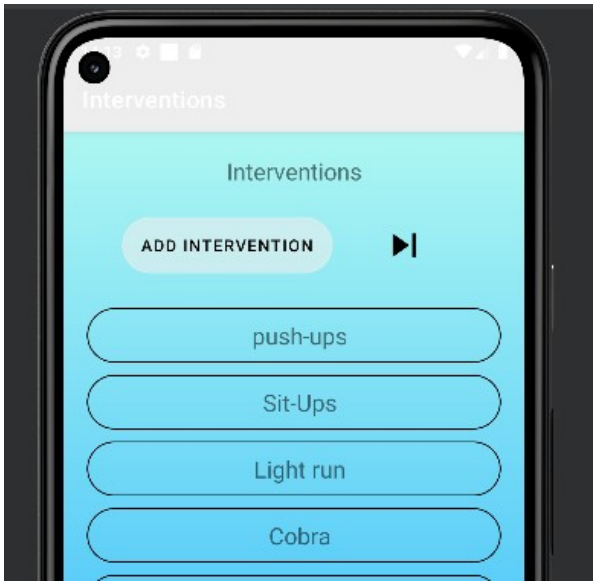
We designed this task to be a focused and relaxing task. We have seen other implementations when googling that show all numbers at once or place a more or less strict time limit on the user, but we felt like it fit the purpose of this task more to have a slow rhythm that encourages a certain amount of concentration while performing it.
When an intervention is scheduled as described before, the pop-up will appear after the digit span task instead of after the questionnaire.

A feature that was not specified in the initial requirements but which we felt would fit the design and intention of the app very well is the Daily Goal. This is a goal that the user can specify him- or herself. A selection is provided by the app, including generally healthy suggestions like drinking a certain amount of water and meditating, but the user is able to enter their own goal as well. The goal is tracked on the home screen via a bar and can be marked as completed. This completion is also displayed in the data screen where the user can see how often they completed each daily task.



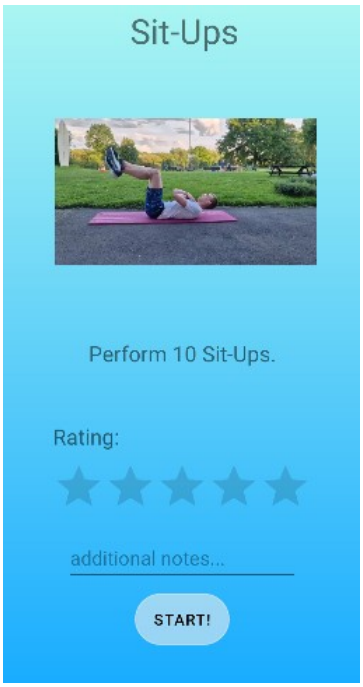
The daily goal also impacts the happiness of our little being „Liftin‘ Larry“ (The namesake of the app). Larry is always present on the home screen and serves as a motivational image in the interventions that do not have a dedicated video. Larry is clickable and a message will pop up that informs the user of Larry‘s current mood. Larry‘s mood is impacted by completing questionnaires and interventions and completing daily goals.





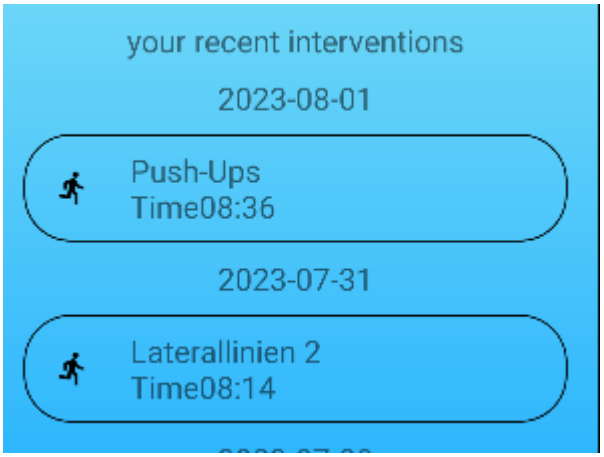
The next most relevant tab in the choice ribbon is the „interventions“ fragment. This is a list of all interventions a user has available to them. These interventions can fall into two categories: Provided interventions and user-defined interventions. We provide a list of 10 interventions. Some of them are relaxing stretching routines and some are more active traditional sports exercises.
In addition, users can use the „Add Intervention“ button to define their own interventions. These will not have a video or description, but they will be tracked in their past interventions and in the data graph. This makes it easily possible to add custom interventions that the user wants to consistently perform. They are also automatically added to the pool of available interventions to be chosen after the questionnaire.

The second button at the top starts an unnamed recording. This is very useful for activities that the user expects to perform only rarely or once, such as spontaneous runs, going climbing, or riding their bike an unknown distance. This will be added to the list of past interventions and the data, but not be stored for re-use.



Clicking on an intervention will open the Single Intervention fragment. Here, the user can see title and description of the intervention, a video demonstrating it and some options to record their own interaction with the intervention. This interaction can happen in three main ways: Firstly, the user can start the intervention, which starts a sensor recording and allows the user to complete it. When completing, the recorded data is saved and the intervention is closed. The user can also rate the intervention, which is then saved for feedback to the researchers, and add additional notes. Both are saved alongside the intervention for later use and analysis.

As mentioned, we save all past interventions the user has performed. This is displayed both in a shortened form on the home screen (as seen in the picture showing „your recent interventions“) and in a longer form in an activity reachable via the „See All Past Activities“ button. Clicking on a performed intervention will open a special data graph which shows the movement recorded during said intervention. This allows a user to track, for example, their elevation change while running.

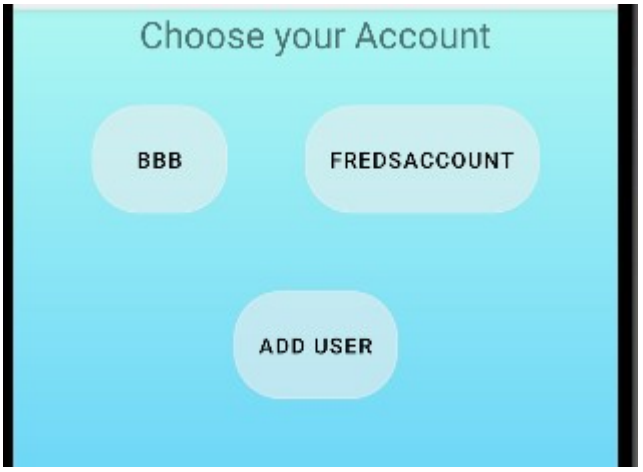


All the previously presented ways of interacting with the app generate some amount of data. This data includes sensor recordings (Like the step count for the daily goal), questionnaire answers and both performed and added interventions. This data is, of course, very valuable to both researchers, who could use it to support their findings, and users, who can track their well-being and their interaction with the app. For this reason, we display it in „Data“ tab on the navigation ribbon.
As is visible on the right, users can see their mental health score over time, their number of interventions and their distribution of tasks in a graph, a bar chart and a pie chart respectively. They can also specify the timeframe they would like to see analysed for them.



There are two more systems contained in the app and both are more structural than functional in nature. Firstly, users can change settings such as the language, permissions, and the timeslots in which they want to receive a notification to complete a questionnaire. In that screen, they can also delete local data or log out from the current account, which is the last system we included.

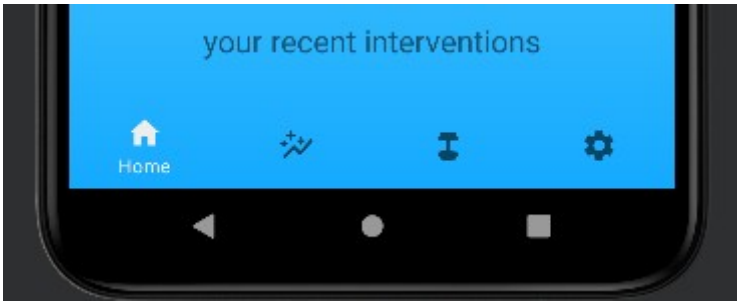
As shown in the screenshot, users can add multiple accounts to their device. These are synchronised through the online component so they carry over data between devices (For a demonstration of this, try to log in with „FredsAccount“ and password „password“!). Users can specify a username and a password. No data is shared between users, allowing for separate intervention lists and data graphs.



Architecture Description (2-3 Pages)

When we first laid out our architectural plans, we envisioned three main components, being the app (frontend), the local backend and the server-side database. This general schema has persisted throughout the app's development and informed much of the development responsibilities.

The app's frontend was originally designed to consist of 4 components: Questionnaire, Settings, Sensor/Interventions and Storage. This has since expanded tremendously. We realised that there needed to be four core states that the user can be in, which can each be an activity: Logging in, navigating the menus, filling out a questionnaire or doing an intervention. We thus created a single activity (the `HomeScreenActivity`) which we could then fill with Fragments implementing all functionality that the user does not explicitly need to commit to. These are the subsystems "Home", "Data", "Interventions" and "Settings". The navigation between these is the first thing the user sees when opening the app and the default way to interact with the app.



Apart from this, the frontend also includes all other activities that the user can enter. What has carried over from the architecture presented in assignment 2 is the Questionnaire, which has stayed the same. The "Sensor/Interventions" component has been split into the "Data" component and the "InterventionList" component, which are accessible through the navigation ribbon. The Intervention List component is also one of the two ways to access the Single Intervention fragment, which represents the state in which the user actively performs an intervention. This was missing entirely from our previous component diagram. Lastly, the Login Activity is the last distinct activity. It is only reachable on first start or through logging out and was also not present at all in assignment 2. It is tied in the closest with the backend, which will be the next topic.

The app's backend included basically only the data saving functionality. This has not changed fundamentally, as data saving and accessing is still the backbone of the app‘s functionality. We have, however added many classes following the „Observer“ pattern such as the AlarmReceiver and the MyLocationListener that provide functionality to interface the user‘s interaction with the app, particularly the interventions, to data generation in the background. To save the data, we still use Android‘s LocalStorage system, as we have diverse data that we need to be able to access and modify and various ways.

The DataSaving helper has expanded from the original implementation in so far as it now also provides many functions to ease interaction with the underlying data. This roughly follows the „Facade“ pattern. For example, the function saveInterventions takes in the

backend representation of a set of interventions (A list of lists of strings) and saves it to the local storage. This avoids exposing the JSON representation of the data and works as a „black box“ that clearly distinguishes between the representations in the backend and in storage. Here, we can see an example of this in action:

```
Vector<Vector<String>> allI = dsHelper.getSavedInterventions( context: HomeScreenActivity.this);
```

The multi-user system is also realised through the backend. By providing different files for each user, we differentiate them both structurally and physically. For each user, we move into an entirely different subsystem of the data storage, allowing users to have varying interventionLists, sensorRecordings and daily goals.

The last major component is our online functionality. This is the second aspect of our implementation of the multi-user requirements, as it allows us to share user data across devices and store it for long-term use. In the ServerHelper, we wrote our own endpoint (Again loosely following the Facade pattern) that allows us to interact with the server backend written in Python and SQL using Java like the rest of the app.

For our backend, we utilize Amazon Web Services with the tools API Gateway, Lambda, and RDS, along with the Retrofit2 android library. The backend serves the sole purpose of data storage and synchronization between various devices, facilitating seamless data exchange with researchers. We use Retrofit2 to send HTTP POST requests from our app to our REST API. Within the body of each request, all locally stored data of a particular type (completed interventions, digit span task, daily goal, etc.) is transmitted, along with the globally unique User ID, generated during the initial sign-up process for each new user. This ID allows us to associate data with individual users while ensuring absolute anonymity, as no personal data such as real names, etc., is stored directly in connection with it. Our REST API then forwards the HTTP requests to a specific script for each data type. This script, running on AWS Lambda with Python 3.10, decodes the message, checks each entry to see if it already exists in the database, and adds any missing data snippets to the database accordingly. Subsequently, all data from the now updated database is retrieved as a JSON object, sorted by timestamp, and returned as the response from the REST API. For this purpose, we use Python's pymysql library. As the database itself, we employ a MySQL-based database within AWS's RDS (Relational Database Service) module. We have seven tables in total, each responsible for storing a particular type of data. Back in the app, it deletes all outdated data that still exists locally and then writes the data present in the response body into the app's local storage. This ensures that the user always has access to their latest data, even across multiple devices, as long as they have an internet connection. The synchronization is automatically triggered after every modification of local data and when a user logs in.

Overall, we kept the fundamental structure of the app being divided into three large components. This division implements two patterns simultaneously: As discussed in the lecture, Android uses the Model-View-Controller pattern by default in the concept of a „View“ and we expanded this by also implementing a two-tiered Client-Server architecture to meet the demands of a Multi-User system. We did adapt the individual components to the changing needs as development went on, such as adapting the pattern for the home

screen, but overall, our prior ideas about how the app should probably need to function were pretty accurate.

Testing Strategy (2-3 Pages)

Thankfully, Android Studio provides a wonderful Test system consisting of UnitTests and InstrumentedTests. These, combined with the default Java Library “JUnit” allow for a very easy to use, but also highly flexible and intricate testing strategy.

Some features, such as the timestamp and the user system could be tested in traditional unit tests, like this one to test if the „User“ class properly validates a password.

```
@Test
public void testCheckPassword() {
    String password = "ipsum";
    User user = new User( localId: 123456,  name: "lorum", password);
    assertTrue(user.checkPassword(password));


}
```

Other classes needed to be tested with instrumented Tests, which proved very useful. The reason for this is that they allow for a “context” to be imported, which allows testing of parts of the app that require with the Android system itself, as will be described later. This allowed us to test components such as the DataSaving Helper:

```
context = InstrumentationRegistry.getInstrumentation().getTargetContext();
...
public void checkGetAllSettingsFromSharedPref(String key, String value) {
    dsHelper.saveSettingAsSharedPref(key, value, context);
...
```

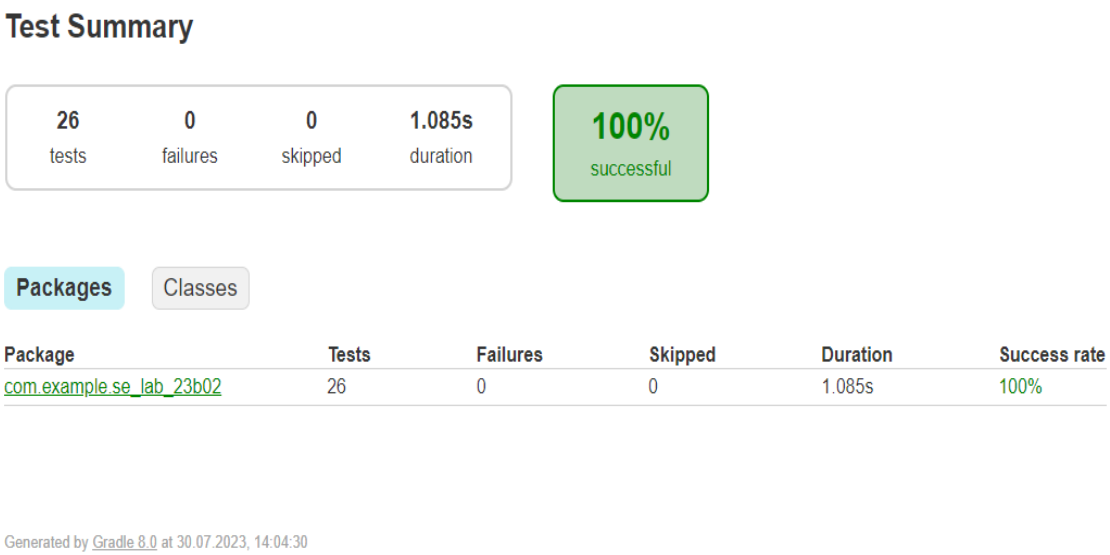
Most frontend functionality did not require much testing except viewing it on different devices with varying screen sizes and ensuring it did not overflow. Especially in the activities which allow user input, like the interventionList and the popup boxes, we made sure that it always looks at least reasonably good.
The backend functionality, though, was run through extensive tests. Sensor recording, data saving, data access, JSON object creation and parsing from different sources, from and to different file formats and many other critical parts of our underlying structure had integration tests written for them to make sure they all worked in unison under as many configurations as possible.

However, a testing system is only as good as the strategy implementing it, and thus we needed to sit down as a group and devise a mode of operation that allows each group member to rely on the fact that the code, they interact with is tested properly.

Because this part of the development process is deeply intertwined with version control, after some deliberation and productive communication in our weekly meetings, we decided to integrate testing right into the GitHub repository itself.
Using GitHub’s workflow system, whenever a group member pushes to the remote repository, the gradle project is built and runs all tests within the folders “project/app/src/test/java/com/example/se_lab_23b02” and “project/app/src/androidTest/java/com/example/se_lab_23b02”. If these tests pass, a small green checkmark indicates that, within the extent of our testing, the current state of the project is operational. This requires each member to ensure that the code they produce is

tested properly. As long as we worked on our own branches, we could also have tests fail without it impacting anyone else, but whenever a pull request was opened, the tests would run and the branch could only be merged if every conflict was resolved. This greatly helped the stability of the main branch: It served, as it should, as a reliable fallback that we could pull from or sync with at any time and be sure it had tested and validated code on it.

Below is pictured an example summary of a test that was performed after a Pull Request:



Unfortunately, because we are using GitHub Enterprise, we the GitHub build pipelines are limited to a certain amount which were exhausted at some point, so we had to run the tests manually again. Because we had no real way to simulate testing in production (as per lecture 04), we handed a late version of our app (approx. 2 weeks before the deadline) to friends and families. We explained the purpose of the app (Which seemed reasonable given it would be used in a research environment) and left them to figure out how to use it. We did not interfere but did take notes on what was intuitive (such as logging in) and what was unintuitive to multiple users (The flow through Questionnaire, DigitSpanTask and Intervention), discussed the patterns we saw and adjusted the app accordingly before testing again.

In doing this, we stuck close to what the lecture recommended: We unit-tested ViewModels, our data layer and the domain layer, as well as utility classes like the dsHelper. UI tests were limited to making sure edge cases are covered (e.g. max-length strings) and every button is accessible and clickable. We made use of mocking through the Android InstrumentationRegistry to provide mock contexts and were surprised just how easy and intuitive it was to use.

Looking back at assignment 3, we took exactly the path envisioned: Individual responsibility to test our own classes and GitHub actions + the Android frameworks to assist us. Rarely, if ever, did anyone need to adjust their code after it was pushed onto the main branch and I cannot recall any instance of a class that was deemed complete not fulfilling its requirements.